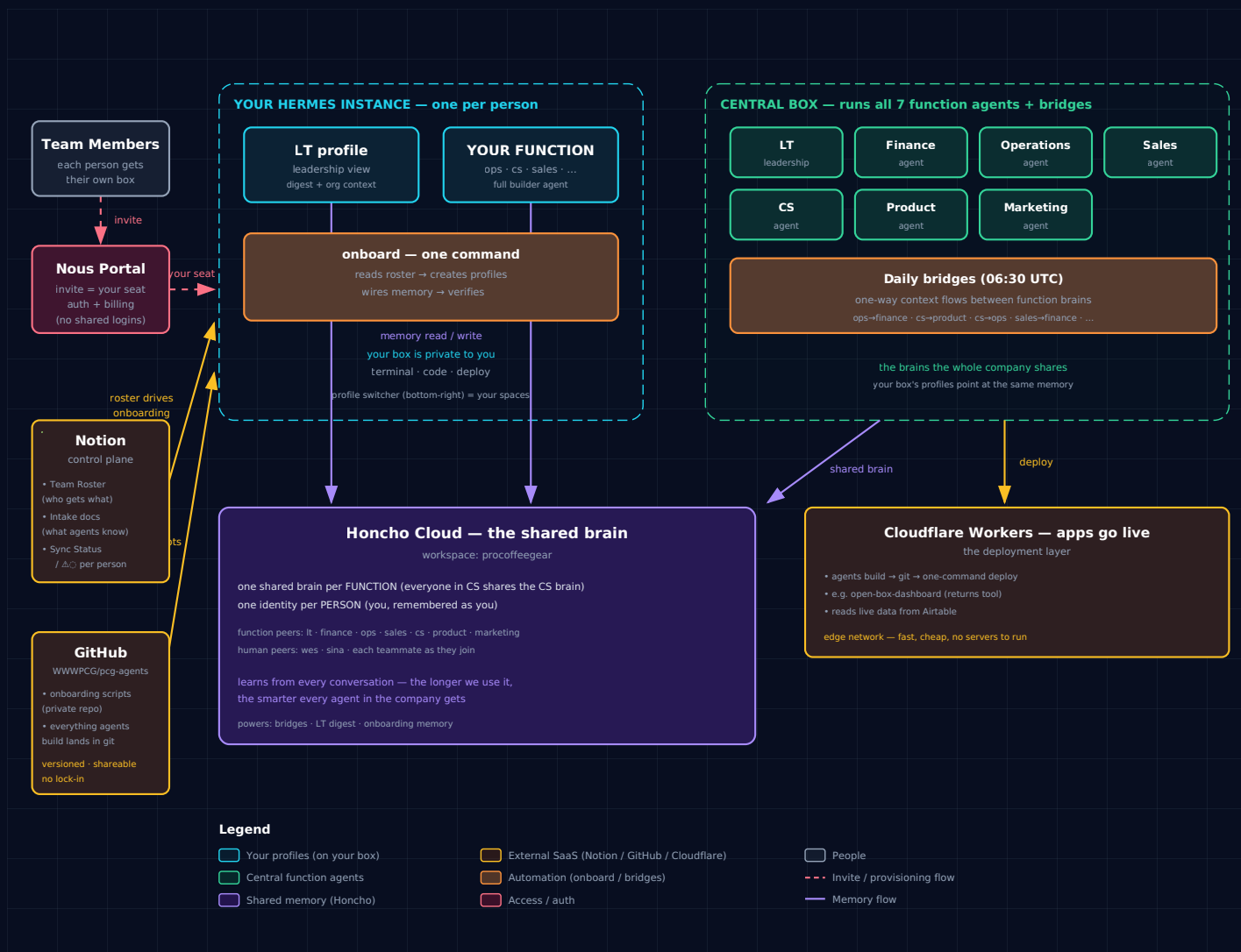


Pro Coffee Gear – Agent Architecture

How our AI agents are organized: one instance per person, one shared brain per function, everything in git.



• What you're looking at

Every person at Pro Coffee Gear gets **their own private agent instance** – your own box, your own login, nobody else can see into it. On your box you run **profiles**: one for each space you belong to. Everyone carries the LT profile (the shared leadership view) plus the profile for your function (Operations, CS, Sales, etc.). You hop between them with the profile switcher, bottom-right of the app.

What makes it more than "everyone has a chatbot" is the layer in the middle: **Honcho, the shared brain**. Each function has exactly one brain, and every person in that function – and the central function agents – read and write the same memory. Your box is private; your function's knowledge is shared. That's the whole trick.

• How it builds context over time

- ▶ **Day one – seeded, not blank.** Each function's agent starts with standing context loaded from its Notion intake doc: who you are, your vendors, your recurring issues, your guardrails. It knows Pro Coffee Gear from the first message.
- ▶ **Every conversation teaches it.** Honcho remembers what matters from your work sessions – decisions, patterns, preferences – and gets sharper the more you use it. This memory survives restarts, upgrades, and box migrations.
- ▶ **Knowledge crosses functions automatically.** Daily bridges move the relevant signal one-way between brains: what CS learns about broken products flows to Product; what Ops learns about vendor costs flows to Finance. No meetings, no re-explaining.
- ▶ **Leadership sees the whole picture.** The LT profile surfaces the cross-functional digest – material changes and decisions-needed – without anyone having to write status reports.

• How you use it in your function

- ▶ **Treat it like a senior colleague who never forgets.** Ask it questions about your area, draft customer replies, analyze data, write docs – it answers with your function's full accumulated knowledge behind it.
- ▶ **Have it build real things.** Your agent has a terminal, can write code, and can deploy. Dashboards, scripts, internal tools, automations – describe what you need, and iterate with it until it works.
- ▶ **Correct it once, benefit forever.** When it gets something wrong, say so. That correction becomes part of the function's memory – the next person never hits the same mistake.

● How it's visible, shared, and learned from

- ▶ **Everything built lands in git.** Code, dashboards, and tools an agent creates live in versioned repos the whole team can inspect, reuse, and improve. Nothing is trapped inside one person's chat history, and nothing locks us to one vendor.
- ▶ **Apps go live on Cloudflare.** When an agent builds something people should actually use – like the returns dashboard – it deploys to Cloudflare Workers and gets a real URL anyone on the team can open. No servers to run, no deploy ceremony.
- ▶ **Function knowledge is inherited.** When someone new joins your function, they plug into the same shared brain on day one – they start where the function is, not from zero.
- ▶ **Wins propagate sideways.** The bridges push one function's learnings to the functions that need them, so a discovery in CS becomes a fix in Product without anyone playing telephone.
- ▶ **Onboarding is self-verifying.** The roster in Notion shows each person's status: Function = what they should have, Active Profiles = what's actually live, and Match / Mismatch tells us instantly if something's off.

● Day one for you

- Accept your Portal invite → your own box
- Run the one onboarding command Wes sends
- Check your roster row: Match + Onboarded ✓
- Open the switcher: LT + your function

● What your agent already knows

- Your function's intake doc (vendors, issues, rules)
- Everything your function's brain has learned
- Cross-functional signal pushed by bridges
- Who you are and how you like to work

● Sharing & learning

- Built things → git repos (inspectable, reusable)
- Live apps → Cloudflare URLs anyone can open
- Function knowledge → shared brain (inherited)
- Learnings → bridges to other functions
- Big picture → LT digest for leadership

● Appendix A – Definitions

- ▶ **Hermes** – the agent platform itself. The app you actually talk to and work in.
- ▶ **Instance (your "box")** – your own private Hermes server. Only you can log into it; nobody else can see inside.
- ▶ **Profile** – one agent identity on a box, with its own memory, tools, and role. You switch profiles bottom-right of the app.
- ▶ **LT profile** – the shared leadership view: company-wide digest and org context. Every team member carries it.
- ▶ **Function profile** – the agent for your team (Operations, CS, Sales...). A full builder: terminal, code, deploy.
- ▶ **Honcho** – the shared memory layer ("the brain"). Lives in the cloud, independent of any box, so knowledge survives restarts and migrations.
- ▶ **Workspace** – our container inside Honcho: `procoffeegear`. All of our peers live here.
- ▶ **Peer** – an identity in Honcho. An AI peer is a function's brain (shared); a human peer is you (personal).
- ▶ **Shared brain** – the one AI peer per function. Everyone in a function reads and writes the same memory.
- ▶ **Standing context** – authored knowledge loaded every session (comes from your function's Notion intake doc).
- ▶ **Learned memory** – what the brain picks up from real conversations over time.
- ▶ **Bridge** – a daily, one-way sync that pushes relevant findings from one function's brain to another (e.g. CS → Product).
- ▶ **Nous Portal** – where seats, billing, and invites are managed. Your invite = your box.
- ▶ **Roster** – the control-plane table in Notion: who gets which functions, plus onboarding verification.
- ▶ **Sync Status** – the roster check comparing what you should have vs. what's actually live. Match = good; Mismatch = tell Wes.
- ▶ **Onboard command** – the one-line setup command that wires a new box to everything and self-verifies.
- ▶ **Git / GitHub** – version control. Everything agents build lands here: visible, reusable, no lock-in.
- ▶ **Cloudflare Workers** – where built apps go live on the internet (e.g. the returns dashboard). Fast, cheap, no servers.
- ▶ **Vendor lock-in** – being stuck with one supplier because leaving is expensive. This stack is designed to avoid it.

● Appendix B – FAQ

Is my box private?

Yes. Your instance is yours alone – nobody else can open it, including Wes. What is shared: work done through your function profile feeds the function's brain, and leadership sees digest-level summaries through LT – never your raw chats.

Do I need to know how to code?

No. Ask in plain English. The agent writes and runs the code; you review the result and steer.

What's the difference between my function profile and LT?

Your function profile is your team's deep expert with full tools. LT is the shared leadership window onto the whole company. Switch between them bottom-right.

Can I peek at another function's agent?

Your box only carries the profiles for your spaces. But you don't lose out – relevant knowledge from other functions reaches your brain automatically through the bridges.

What if the agent gets something wrong?

Tell it, plainly. Corrections become permanent function memory – you fix it once, for everyone, forever.

Where does the stuff I build end up?

Code and tools land in git repos the team can inspect and reuse. Anything worth using daily gets deployed to Cloudflare with a URL anyone can open.

My roster row shows Mismatch after onboarding. Now what?

Don't retry the command. Take a screenshot of your row and send it to Wes – a mismatch means something specific failed and he'll fix it.

Who pays for this?

The company. Your seat is covered under the team plan – you'll never see a bill.

Does our company data train public AI models?

Our memory lives in our own private Honcho workspace, and our code in private repos. We deliberately chose a stack we control – that's the anti-lock-in design.

What happens when someone changes roles or leaves?

Their roster row gets updated and their box is rewired or retired. The function brain keeps everything the function learned – knowledge stays with the company, not the person.